Django 6.0 — Complete Bangla Notes

1. Django How-to Guides

Django How-to Guides হলো Practical Documentation, যেখানে Django ব্যবহার করে বাস্তব সমস্যার সমাধান করার পদ্ধতি শেখানো হয়।

Official Documentation:
https://docs.djangoproject.com/en/6.0/howto/

1.1 How to Provide Initial Data for Models

Initial Data হলো Application শুরু করার সময় Database-এ আগে থেকে থাকা প্রয়োজনীয় Data।

Django-তে Initial Data দেওয়ার জন্য Fixture ব্যবহার করা যায়।

Fixture হলো JSON, XML অথবা YAML Format-এ সংরক্ষিত Data।

```bash
python manage.py dumpdata myapp.Student > students.json
python manage.py loaddata students.json
```

এখানে:
- `dumpdata` — Database-এর Data Export করে।
- `loaddata` — Fixture থেকে Database-এ Data Import করে।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/initial-data/

1.2 How to Integrate Django with a Legacy Database

Legacy Database হলো আগে থেকে তৈরি এবং ব্যবহৃত Database।

Django-এর `inspectdb` Command দিয়ে Existing Database-এর Table থেকে Model তৈরি করা যায়।

```bash
python manage.py inspectdb
```

```

একটি File-এ সংরক্ষণ করতে:

```bash
python manage.py inspectdb > models.py
```

Generated Model-গুলো Review করে Django App-এর উপযুক্ত `models.py` ফাইলে রাখতে হবে।

Existing Table-এর জন্য Django সাধারণত `managed = False` ব্যবহার করতে পারে। এর অর্থ Django Migration দিয়ে Table পরিচালনা করবে না।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/legacy-databases/

1.3 How to Create Custom Model Fields

Django-তে প্রয়োজন অনুযায়ী Custom Model Field তৈরি করা যায়।

```python
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()
```

Built-in Field যথেষ্ট না হলে Custom Field তৈরি করা যায়।

Custom Field তৈরি করার সময় সাধারণত `models.Field` অথবা উপযুক্ত Built-in Field থেকে Inherit করা হয়।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/custom-model-fields/

1.4 How to Create Database Migrations

Migration হলো Model-এর পরিবর্তন Database Schema-তে প্রয়োগ করার ব্যবস্থা।
```

```bash
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
```

- `makemigrations` — Model পরিবর্তন থেকে Migration File তৈরি করে।
- `migrate` — Migration Database-এ Apply করে।
- `showmigrations` — Migration-এর Status দেখায়।

Documentation:
https://docs.djangoproject.com/en/6.0/topics/migrations/

1.5 How to Write Custom Lookups

Django ORM-এ Custom Lookup তৈরি করে Query Filter-এর নতুন Behaviour যোগ করা যায়।

উদাহরণ:

```python
Student.objects.filter(name__exact="Rahim")
```

এখানে `exact` একটি Built-in Lookup।

Custom Lookup তৈরি করতে `Lookup` Class ব্যবহার করা যায়।

```python
from django.db.models import Lookup, CharField


@CharField.register_lookup
class NotEqual(Lookup):
    lookup_name = "ne"

    def as_sql(self, compiler, connection):
        lhs, lhs_params = self.process_lhs(
            compiler, connection
        )
        rhs, rhs_params = self.process_rhs(
            compiler, connection
```

```
        )

        params = lhs_params + rhs_params

        return f"{lhs} <> {rhs}", params
```

ব্যবহার:

```python
Student.objects.filter(name__ne="Rahim")
```

এই উদাহরণটি সাধারণ SQL তুলনা দেখায়। বাস্তবে NULL Value এবং Database Backend-এর আচরণ বিবেচনা করতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/custom-lookups/

1.6 How to Generate CSV and PDF Files

Python-এর `csv` Module ব্যবহার করে CSV File তৈরি করা যায়।

```python
import csv

from django.http import HttpResponse


def export_csv(request):
    response = HttpResponse(
        content_type="text/csv"
    )
    response["Content-Disposition"] = (
        'attachment; filename="students.csv"'
    )

    writer = csv.writer(response)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Rahim", 20])
    writer.writerow(["Karim", 22])
```

```
    return response
```

PDF তৈরি করতে ReportLab-এর মতো Library ব্যবহার করা যায়।

```bash
pip install reportlab
```

PDF তৈরি করার সময় বাংলা Text সঠিকভাবে দেখাতে উপযুক্ত Bengali Unicode Font Register করতে হতে পারে।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/outputting-csv/

1.7 How to Override Templates

Django Template Override করে Built-in অথবা Third-Party App-এর Template পরিবর্তন করা যায়।

উদাহরণ:

```text
templates/
    admin/
        base_site.html
```

`settings.py`-এ Template Directory Configure করা যেতে পারে।

```python
TEMPLATES = [
    {
        "BACKEND": (
            "django.template.backends.django.DjangoTemplates"
        ),
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
```

```
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/overriding-templates/

1.8 How to Write a Custom Authentication Backend

Custom Authentication Backend ব্যবহার করে নিজের Authentication Logic তৈরি করা যায়।

```python
from django.contrib.auth.backends import BaseBackend


class CustomBackend(BaseBackend):
    def authenticate(self, request, username=None, password=None):
        # এখানে Authentication Logic লিখতে হবে
        return None

    def get_user(self, user_id):
        return None
```

`settings.py`:

```python
AUTHENTICATION_BACKENDS = [
    "myapp.backends.CustomBackend",
]
```

এটি একটি Skeleton Example। বাস্তবে User Verify করা এবং নিরাপদ Password Handling যুক্ত করতে হবে।

Documentation:

https://docs.djangoproject.com/en/6.0/topics/auth/customizing/

1.9 How to Create Custom Template Tags and Filters

Custom Template Filter তৈরি করতে `templatetags` Directory ব্যবহার করা যায়।

Project Structure:

```text
myapp/
    templatetags/
        __init__.py
        custom_tags.py
```

`custom_tags.py`:

```python
from django import template

register = template.Library()


@register.filter
def upper_text(value):
    return value.upper()
```

Template-এ ব্যবহার:

```django
{% load custom_tags %}

{{ "hello"|upper_text }}
```

Output:

```text
HELLO
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/custom-template-tags/

1.10 How to Manage Static Files

Static Files হলো CSS, JavaScript, Image ইত্যাদি।

`settings.py`:

```python
STATIC_URL = "static/"
```

Template-এ ব্যবহার:

```django
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Production Deployment-এর সময় Static Files Collect করতে:

```bash
python manage.py collectstatic
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/static-files/

1.11 How to Configure Logging

Logging ব্যবহার করে Application-এর Event, Warning এবং Error Record করা যায়।

```python
import logging

logger = logging.getLogger(__name__)
```

```
def my_view(request):
    logger.info("View accessed")
```

গুরুত্বপূর্ণ Logging Level:

- `DEBUG`
- `INFO`
- `WARNING`
- `ERROR`
- `CRITICAL`

Documentation:
https://docs.djangoproject.com/en/6.0/topics/logging/

1.12 How to Configure Error Reporting

Production Application-এ Error Reporting গুরুত্বপূর্ণ।

`DEBUG = False` ব্যবহার করলে Error Page-এ Sensitive Information প্রকাশের ঝুঁকি কমে।

```python
DEBUG = False
```

Production-এ `ALLOWED_HOSTS` সঠিকভাবে Configure করতে হবে।

```python
ALLOWED_HOSTS = [
    "example.com",
    "www.example.com",
]
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/error-reporting/

1.13 How to Uninstall an Application

কোনো App Project থেকে সরানোর আগে তার Dependency এবং Data বিবেচনা করতে হবে।

সাধারণ প্রক্রিয়া:

1. App-এর ব্যবহার এবং Dependency পরীক্ষা করা।
2. প্রয়োজনীয় Data Backup নেওয়া।
3. `INSTALLED_APPS` থেকে App সরানো।
4. প্রয়োজন হলে URL, Import এবং Related Configuration সরানো।
5. Database Table বা Migration পরিবর্তনের আগে সতর্কভাবে পরিকল্পনা করা।

শুধু `INSTALLED_APPS` থেকে App সরালেই তার Database Table স্বয়ংক্রিয়ভাবে Delete হয়ে যায় না।

Documentation:
https://docs.djangoproject.com/en/6.0/

1.14 How to Upgrade Django

Django Upgrade করার আগে Release Notes এবং Compatibility পরীক্ষা করতে হবে।

Version পরীক্ষা:

```bash
python -m django --version
```

Installed Package Upgrade:

```bash
python -m pip install --upgrade Django
```

Upgrade করার আগে:
- Project Backup নিতে হবে।
- Dependency Compatibility পরীক্ষা করতে হবে।
- Release Notes পড়তে হবে।
- Test Suite চালাতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/releases/

1.15 How to Use Django on Windows

Virtual Environment তৈরি:

```powershell
python -m venv .venv
```

Activate:

```powershell
.venv\Scripts\Activate.ps1
```

Django Install:

```powershell
python -m pip install Django
```

Project তৈরি:

```powershell
django-admin startproject config .
python manage.py runserver
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/windows/

1.16 How to Deploy Django

Production-এ Django Deploy করার আগে নিচের বিষয়গুলো Configure করতে হবে।

```python
DEBUG = False
```

```python
ALLOWED_HOSTS = ["example.com"]
```

Deployment Checklist:

- Production Database Configure করা।
- Secret Key Environment Variable-এ রাখা।
- HTTPS Configure করা।
- Static Files Serve করার ব্যবস্থা করা।
- Media Files-এর Storage Configure করা।
- Error Logging Configure করা।
- Security Settings পরীক্ষা করা।
- Deployment Checklist চালানো।

```bash
python manage.py check --deploy
```

Django-এর Development Server Production-এর জন্য উপযুক্ত নয়।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/deployment/

1.17 How to Use REMOTE_USER Authentication

কোনো Trusted Web Server যদি Authentication করে এবং `REMOTE_USER` সরবরাহ করে, তাহলে Django-তে Remote User Authentication Configure করা যায়।

```python
MIDDLEWARE = [
    # অন্যান্য Middleware
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.auth.middleware.RemoteUserMiddleware",
]
```

```python
AUTHENTICATION_BACKENDS = [
    "django.contrib.auth.backends.RemoteUserBackend",
]
```

এটি Configure করার সময় নিশ্চিত করতে হবে যে Remote User Header বা Environment Value অবিশ্বস্ত Client সরাসরি পাঠাতে না পারে।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/auth-remote-user/

1.18 Content Security Policy (CSP)

Content Security Policy Browser-কে কোন Source থেকে Resource Load করা যাবে তা নির্ধারণ করতে সাহায্য করে।

CSP-এর উদ্দেশ্য হলো কিছু ধরনের Cross-Site Scripting (XSS) Attack-এর ঝুঁকি কমানো।

Django 6.0-এর CSP Support এবং Settings সম্পর্কে Official Documentation দেখতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/

1.19 Cross-Site Request Forgery (CSRF)

CSRF হলো এমন Attack যেখানে Authenticated User-এর Browser ব্যবহার করে অনিচ্ছাকৃত Request পাঠানো হয়।

Django Template Form-এ সাধারণত CSRF Token ব্যবহার করা হয়।

```django
<form method="post">
    {% csrf_token %}
    <button type="submit">Submit</button>
</form>
```

POST Request-এর ক্ষেত্রে CSRF Protection গুরুত্বপূর্ণ।

Documentation:
https://docs.djangoproject.com/en/6.0/howto/csrf/

1.20 How to Create a Custom Storage System

Django Storage API ব্যবহার করে Custom File Storage Backend তৈরি করা যায়।

এটি Local Storage, Cloud Storage বা অন্য কোনো Storage System-এর সঙ্গে Integration করতে সাহায্য করে।

Custom Storage Implement করার আগে Django-এর Storage API এবং সংশ্লিষ্ট Interface সম্পর্কে জানতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/files/storage/

1.21 How to Write Custom Management Commands

Custom Management Command দিয়ে Terminal থেকে নিজের Application-এর কাজ চালানো যায়।

Project Structure:

```text
myapp/
    management/
        __init__.py
        commands/
            __init__.py
            hello.py
```

`hello.py`:

```python
from django.core.management.base import BaseCommand


class Command(BaseCommand):
    help = "Print a hello message"

    def handle(self, *args, **options):
        self.stdout.write(
            self.style.SUCCESS("Hello Django!")
        )
```

Run:

```bash
python manage.py hello
```

Documentation:
https://docs.djangoproject.com/en/6.0/howto/custom-management-commands/

1.22 How to Use Django Shell

Django Shell ব্যবহার করে Interactive Mode-এ Model, Query এবং Application Code পরীক্ষা করা যায়।

```bash
python manage.py shell
```

উদাহরণ:

```python
from myapp.models import Student

Student.objects.all()
Student.objects.create(name="Rahim", age=20)
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/django-admin/#shell

---

2. Django FAQ

FAQ-এর পূর্ণরূপ হলো Frequently Asked Questions।

Official Documentation:
https://docs.djangoproject.com/en/6.0/faq/

2.1 General FAQ

Django কী?

Django হলো Python-এর একটি High-Level Web Framework। এটি দ্রুত, নিরাপদ এবং Maintainable Web Application তৈরি করতে সাহায্য করে।

Django-এর Architecture কী?

Django সাধারণত Model-Template-View (MTV) Architecture অনুসরণ করে।

- Model – Data এবং Database-এর সঙ্গে কাজ করে।
- Template – User Interface তৈরি করে।
- View – Request Handle করে এবং Response তৈরি করে।
- URLconf – URL-এর সঙ্গে View-এর Mapping করে।

Django কি Free?

হ্যাঁ, Django একটি Free এবং Open-Source Framework।

2.2 Installation FAQ

Django Install:

```bash
python -m pip install Django
```

Version Check:

```bash
python -m django --version
```

Project তৈরি:

```bash
django-admin startproject config
```

Development Server:

```bash
cd config
python manage.py runserver
```

2.3 Usage FAQ

App তৈরি:

```bash
python manage.py startapp students
```

`settings.py`-এ App Register:

```python
INSTALLED_APPS = [
    # Django-এর Built-in Apps
    "students",
]
```

Migration তৈরি ও Apply:

```bash
python manage.py makemigrations
python manage.py migrate
```

Superuser তৈরি:

```bash
python manage.py createsuperuser
```

2.4 Model FAQ

Model হলো Python Class, যা Database-এর Data Structure এবং Behaviour নির্ধারণ করে।

```python
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()
```

```
    def __str__(self):
        return self.name
```

Query:

```python
Student.objects.all()
Student.objects.filter(age__gte=18)
Student.objects.get(id=1)
```

2.5 Admin FAQ

Django Admin হলো Database Data Manage করার জন্য Built-in Interface।

`admin.py`:

```python
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

Superuser তৈরি:

```bash
python manage.py createsuperuser
```

Default Admin URL সাধারণত:

```text
http://127.0.0.1:8000/admin/
```

2.6 Troubleshooting FAQ

Migration Error হলে কী করব?

```bash
python manage.py makemigrations
python manage.py migrate
```

Migration File, Database Configuration এবং Error Message পরীক্ষা করতে হবে।

ModuleNotFoundError হলে কী করব?

- Virtual Environment Activate হয়েছে কি না দেখুন।
- প্রয়োজনীয় Package Install হয়েছে কি না পরীক্ষা করুন।
- Import Path সঠিক কি না দেখুন।

TemplateDoesNotExist হলে কী করব?

- Template File-এর নাম পরীক্ষা করুন।
- Template Directory সঠিক কি না দেখুন।
- `TEMPLATES` Setting পরীক্ষা করুন।
- App-এর Template Structure সঠিক কি না দেখুন।

DisallowedHost Error হলে কী করব?

`ALLOWED_HOSTS`-এ প্রয়োজনীয় Host Name যুক্ত করতে হবে।

---

3. Django API Reference

API Reference হলো Django-এর Classes, Functions, Modules, Settings এবং Methods-এর বিস্তারিত Technical Documentation।

Official Documentation:
https://docs.djangoproject.com/en/6.0/ref/

3.1 Applications

Django Application হলো Project-এর একটি Modular অংশ।

App তৈরি:

```bash
python manage.py startapp blog
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/applications/

3.2 System Checks

Django System Check দিয়ে Project-এর Configuration এবং সম্ভাব্য সমস্যা পরীক্ষা করা যায়।

```bash
python manage.py check
```

Production Check:

```bash
python manage.py check --deploy
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/checks/

3.3 Class-Based Views

Class-Based View (CBV) হলো Class ব্যবহার করে View তৈরি করার পদ্ধতি।

```python
from django.views.generic import ListView
from .models import Student


class StudentListView(ListView):
    model = Student
    template_name = "students/list.html"
    context_object_name = "students"
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/class-based-views/

3.4 Clickjacking Protection

Clickjacking হলো এমন Attack যেখানে User-কে বিভ্রান্ত করে অন্য Page-এর উপর অদৃশ্য বা বিভ্রান্তিকর UI ব্যবহার করানো হয়।

Django-এর Clickjacking Protection Middleware ব্যবহার করা যায়।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/clickjacking/

3.5 Django Contrib

`django.contrib`-এ Django-এর বিভিন্ন Built-in Application রয়েছে।

উদাহরণ:

- `django.contrib.admin`
- `django.contrib.auth`
- `django.contrib.contenttypes`
- `django.contrib.sessions`
- `django.contrib.messages`
- `django.contrib.staticfiles`

Documentation:
https://docs.djangoproject.com/en/6.0/ref/contrib/

3.6 Cross-Site Request Forgery (CSRF)

Django CSRF Protection Middleware এবং Token-এর মাধ্যমে CSRF Attack প্রতিরোধে সাহায্য করে।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/csrf/

3.7 Databases

Django ORM-এর মাধ্যমে Python Code ব্যবহার করে Database-এর সঙ্গে কাজ করা যায়।

```python
Student.objects.create(name="Rahim", age=20)
```

```
Student.objects.filter(age__gte=18)

Student.objects.order_by("name")
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/databases/

3.8 django-admin and manage.py

Django-এর Administrative Commands চালানোর জন্য `django-admin` এবং `manage.py` ব্যবহার করা হয়।

```bash
python manage.py runserver
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py check
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/django-admin/

3.9 Exceptions

Django বিভিন্ন ধরনের Exception ব্যবহার করে।

উদাহরণ:

```python
from django.http import Http404
from django.shortcuts import get_object_or_404

student = get_object_or_404(Student, pk=1)
```

`get_object_or_404()` Object না পেলে HTTP 404 Response তৈরি করে।

Documentation:

https://docs.djangoproject.com/en/6.0/ref/exceptions/

3.10 File Handling

Django Model-এ File এবং Image সংরক্ষণ করতে `FileField` ও `ImageField` ব্যবহার করা যায়।

```python
from django.db import models


class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to="documents/")
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/files/

3.11 Forms

Django Forms User Input Validate এবং Process করতে সাহায্য করে।

```python
from django import forms


class StudentForm(forms.Form):
    name = forms.CharField(max_length=100)
    age = forms.IntegerField(min_value=1)
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/forms/

3.12 Logging

Logging Application-এর Event এবং Error Record করতে ব্যবহৃত হয়।

```python
import logging
```

```
logger = logging.getLogger(__name__)

logger.info("Application started")
logger.warning("This is a warning")
logger.error("An error occurred")
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/logging/

3.13 Middleware

Middleware হলো Request এবং Response Processing-এর মাঝখানে কাজ করা Component।

সাধারণ Middleware Flow:

```text
Request
   |
Middleware
   |
View
   |
Middleware
   |
Response
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/middleware/

3.14 Migration Operations

Migration Operations Database Schema পরিবর্তন করে।

উদাহরণ:

- `CreateModel`
- `DeleteModel`
- `AddField`

- `RemoveField`
- `AlterField`
- `RenameField`

Documentation:
https://docs.djangoproject.com/en/6.0/ref/migration-operations/

3.15 Models

Django Model Database-এর Table এবং Data Represent করে।

```python
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/models/

3.16 Paginator

Paginator বড় Data-কে একাধিক Page-এ ভাগ করতে সাহায্য করে।

```python
from django.core.paginator import Paginator

students = Student.objects.all()

paginator = Paginator(students, 10)
page_obj = paginator.get_page(1)
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/paginator/

3.17 Request and Response

Django View সাধারণত Request গ্রহণ করে এবং Response Return করে।

```python
from django.http import HttpResponse



def home(request):
    return HttpResponse("Hello Django")
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/request-response/

3.18 Schema Editor

Schema Editor Database Table, Field এবং Schema পরিবর্তনের কাজে ব্যবহৃত হয়। এটি সাধারণত Migration Framework-এর মাধ্যমে ব্যবহৃত হয়।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/schema-editor/

3.19 Django Settings

`settings.py`-এ Project-এর Configuration রাখা হয়।

উদাহরণ:

```python
DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = []

MIDDLEWARE = []
```

Production-এ `DEBUG = False` এবং উপযুক্ত `ALLOWED_HOSTS` Configure করতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/settings/

3.20 Signals

Django Signals কোনো নির্দিষ্ট Event ঘটলে অন্য অংশে Notification পাঠাতে সাহায্য করে।

উদাহরণ:

```python
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Student



@receiver(post_save, sender=Student)
def student_saved(sender, instance, created, **kwargs):
    if created:
        print("New student created")
```

Signals ব্যবহার করার সময় অপ্রয়োজনীয় জটিলতা তৈরি হচ্ছে কি না বিবেচনা করতে হবে।

Documentation:
https://docs.djangoproject.com/en/6.0/topics/signals/

3.21 Tasks

Background Task-এর জন্য Django Project-এ আলাদা Task Processing System ব্যবহার করা যেতে পারে।

Email Sending, Report Generation এবং দীর্ঘ সময়ের কাজ Background-এ চালানোর ক্ষেত্রে Task Queue উপকারী হতে পারে।

Documentation:
https://docs.djangoproject.com/en/6.0/

3.22 Templates

Django Template Engine HTML-এর মধ্যে Dynamic Data দেখাতে সাহায্য করে।

```django
<h1>Hello, {{ name }}</h1>
```

```
{% if students %}
    {% for student in students %}
        <p>{{ student.name }}</p>
    {% endfor %}
{% endif %}
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/templates/

3.23 TemplateResponse

`TemplateResponse` Template Render করার জন্য Response Object তৈরি করে।

```python
from django.template.response import TemplateResponse


def home(request):
    return TemplateResponse(
        request,
        "home.html",
        {"name": "Rahim"},
    )
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/template-response/

3.24 Unicode

Python String এবং Django Template Unicode Text নিয়ে কাজ করতে পারে।

বাংলা Text-এর উদাহরণ:

```python
message = "আমি Django শিখছি"
```

Documentation:

https://docs.djangoproject.com/en/6.0/ref/unicode/

3.25 URL Resolver

URL Resolver Incoming URL-কে উপযুক্ত View-এর সঙ্গে Match করে।

`urls.py`:

```python
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/urlresolvers/

3.26 Django Utilities

Django Utilities-এ বিভিন্ন Helper Function এবং Utility Class রয়েছে।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/utils/

3.27 Validators

Validator কোনো Value নির্দিষ্ট শর্ত পূরণ করে কি না তা পরীক্ষা করে।

```python
from django.core.validators import MinValueValidator
from django.db import models


class Product(models.Model):
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0)],
```

```
    )
```

Documentation:
https://docs.djangoproject.com/en/6.0/ref/validators/

3.28 Built-in Views

Django কিছু Built-in View এবং Generic View সরবরাহ করে, যা সাধারণ Web Development Task সহজ করে।

Documentation:
https://docs.djangoproject.com/en/6.0/ref/views/

---

4. Django Meta-documentation

Meta-documentation হলো Django Documentation কীভাবে লেখা, সংগঠিত এবং অবদান রাখা হয় তার Documentation।

এখানে সাধারণত Documentation Style, Contribution এবং Documentation Build করার বিষয় আলোচনা করা হয়।

Official Documentation:
https://docs.djangoproject.com/en/6.0/

4.1 Documentation Contribution

Django Open-Source Project হওয়ায় Documentation উন্নত করার জন্য Contribution করা যায়।

Contribution করার আগে:
- Existing Documentation পড়তে হবে।
- Contribution Guidelines অনুসরণ করতে হবে।
- প্রয়োজনীয় Test এবং Documentation Check চালাতে হবে।
- Pull Request-এর মাধ্যমে পরিবর্তন জমা দিতে হবে।

Django Project:
https://github.com/django/django

---

5. Django Internationalization and Translation

Internationalization-এর সংক্ষিপ্ত রূপ হলো **i18n**।

Localization-এর সংক্ষিপ্ত রূপ হলো **l10n**।

- Internationalization – Application-কে একাধিক Language এবং Locale সমর্থনের উপযোগী করা।
- Localization – নির্দিষ্ট Language, Date Format, Number Format ইত্যাদির জন্য Application সাজানো।

Official Documentation:
https://docs.djangoproject.com/en/6.0/topics/i18n/

5.1 Translation Setup

`settings.py`:

```python
USE_I18N = True

LANGUAGE_CODE = "en-us"

LANGUAGES = [
    ("en", "English"),
    ("bn", "Bengali"),
]
```

5.2 Translate Text in Python

```python
from django.utils.translation import gettext as _


def home(request):
    message = _("Welcome to our website")
```

`gettext()` ব্যবহার করলে Text Translation-এর জন্য Mark করা হয়।

5.3 Translate Text in Templates

```django
{% load i18n %}

<h1>{% trans "Welcome" %}</h1>
```

একাধিক Text-এর জন্য:

```django
{% load i18n %}

{% blocktrans %}
Welcome to our website.
{% endblocktrans %}
```

5.4 Create Translation Files

Translation File তৈরি করতে:

```bash
django-admin makemessages -l bn
```

Translation Compile করতে:

```bash
django-admin compilemessages
```

এই Command চালানোর জন্য প্রয়োজনীয় Translation Tools এবং Environment Configure করা থাকতে হবে।

5.5 Language Prefix in URLs

Django-এর `i18n_patterns()` ব্যবহার করে URL-এ Language Prefix যোগ করা যায়।

```python
from django.conf.urls.i18n import i18n_patterns
from django.urls import path
from . import views

urlpatterns = i18n_patterns(
    path("", views.home, name="home"),
)
```

Configured Language এবং URL Configuration অনুযায়ী URL-এ Language Prefix যুক্ত হতে পারে।

Documentation:
https://docs.djangoproject.com/en/6.0/topics/i18n/translation/

---

6. Important Django Commands Cheat Sheet

```bash
# Create Project
django-admin startproject config

# Create App
python manage.py startapp students

# Run Development Server
python manage.py runserver

# Create Migrations
python manage.py makemigrations

# Apply Migrations
python manage.py migrate

# Show Migrations
python manage.py showmigrations

# Create Superuser
python manage.py createsuperuser
```

```
# Open Django Shell
python manage.py shell

# Run System Checks
python manage.py check

# Run Production Deployment Checks
python manage.py check --deploy

# Collect Static Files
python manage.py collectstatic

# Inspect Existing Database
python manage.py inspectdb

# Export Database Data
python manage.py dumpdata

# Import Fixture Data
python manage.py loaddata

# Run Tests
python manage.py test

# Display Django Version
python -m django --version
```

---

7. Django Learning Checklist

Django ভালোভাবে শিখতে নিচের বিষয়গুলো Practice করা উচিত।

- [ ] Django Project এবং App তৈরি
- [ ] URL এবং View
- [ ] Templates
- [ ] Models এবং ORM
- [ ] Migrations
- [ ] Django Admin

```
- [ ] Forms এবং Validation
- [ ] Authentication এবং Authorization
- [ ] Static Files এবং Media Files
- [ ] Middleware
- [ ] Signals
- [ ] Class-Based Views
- [ ] Pagination
- [ ] File Upload
- [ ] CSV Export
- [ ] Custom Management Commands
- [ ] Custom Model Fields
- [ ] Custom Template Tags
- [ ] Custom Lookups
- [ ] Logging এবং Error Handling
- [ ] Testing
- [ ] Security
- [ ] Internationalization
- [ ] Database Integration
- [ ] Production Deployment

---

## 8. Official Django 6.0 Documentation Links

| Topic | Documentation |
|---|---|
| Django Overview | https://docs.djangoproject.com/en/6.0/ |
| How-to Guides | https://docs.djangoproject.com/en/6.0/howto/ |
| FAQ | https://docs.djangoproject.com/en/6.0/faq/ |
| API Reference | https://docs.djangoproject.com/en/6.0/ref/ |
| Models | https://docs.djangoproject.com/en/6.0/topics/db/models/ |
| QuerySets | https://docs.djangoproject.com/en/6.0/ref/models/querysets/ |
| Forms | https://docs.djangoproject.com/en/6.0/topics/forms/ |
| Authentication | https://docs.djangoproject.com/en/6.0/topics/auth/ |
| Templates | https://docs.djangoproject.com/en/6.0/topics/templates/ |
| URL Dispatcher | https://docs.djangoproject.com/en/6.0/topics/http/urls/ |
| Migrations | https://docs.djangoproject.com/en/6.0/topics/migrations/ |
| Testing | https://docs.djangoproject.com/en/6.0/topics/testing/ |
```

```
| Security | https://docs.djangoproject.com/en/6.0/topics/security/ |
| Deployment | https://docs.djangoproject.com/en/6.0/howto/deployment/ |
| Internationalization |
https://docs.djangoproject.com/en/6.0/topics/i18n/ |

**শেষ কথা:** Django শেখার সময় শুধু Documentation পড়লেই হবে না। প্রতিটি Topic ছোট Project-এ Implement করে Practice করতে হবে। বিশেষ করে Models, ORM, Authentication, Forms, Security, Testing এবং Deployment ভালোভাবে শেখা জরুরি।
```